

SIN GENERADOR — 100% A MANO

Escribir el código a mano, paso a paso

Cómo crear una prueba escribiendo cada archivo tú mismo, siguiendo los patrones del framework.

1

Datos + interface

jsonData + core/interfaces

2

Page Object

src/pages

3

Step Definitions

src/test/stepsDefinitions

4

Feature

src/test/features

Panorama: las capas y el orden

Una prueba se compone de 4 archivos que se conectan entre sí. Escribirlos a mano es entender cómo encajan. El generador hace esto mismo por ti; aquí lo haces tú.

Las 4 capas y dónde vive cada una

Capa	Archivo	Qué contiene
Feature	src/test/features/<mod>/<Mod>.feature	El escenario en lenguaje Gherkin (Given/When/Then)
Steps	src/test/stepsDefinitions/<mod>/<Mod>StepDefinitions.ts	Conecta cada frase con métodos de la página
Page Object	src/pages/<Mod>Page.ts	Locators y métodos de acción de la pantalla
Datos	jsonData/<env>/<mod>.json (+ interface)	Los datos de prueba

Cómo fluye una ejecución

Feature: una línea del escenario

when ingresa el usuario "valido"



Step Definition: la vincula

this.getPage(LoginPage).fillUsername(...)



Page Object: tu método

fillField(this.usernameInput, value)



BasePage: acción + evidencia

locator.fill(value) + screenshot



INFO — Orden recomendado para escribir a mano

De abajo hacia arriba: 1) datos + interface, 2) Page Object, 3) Step Definitions, 4) Feature. Así, cuando escribes cada capa, la de abajo ya existe.

1

Datos + interface

Los datos de prueba van en un JSON, y su "forma" (tipos) en una interface TypeScript. Los datos se separan de la lógica para poder reutilizarlos.

a) El archivo de datos

Crea jsonData/qa/login.json (un array de registros; cada uno con id único):

```
jsonData/qa/login.json
```

```
[  
  { "id": "valido", "username": "Admin", "password": "admin123" }  
]
```

b) La interface

Crea core/interfaces/LoginData.ts. El id es obligatorio; agrega un campo por cada propiedad de tus datos:

```
core/interfaces/LoginData.ts
```

```
export interface LoginData {  
  id: string;  
  username: string;  
  password: string;  
}
```



CONSEJO — ¿Por qué la interface?

Da autocompletado y evita errores: cuando en el step escribas data.username, TypeScript verifica que ese campo existe. JsonDataManagement.getById<LoginData>(…) devuelve un objeto con estos tipos.

2 Page Object

Representa una pantalla: sus locators (elementos) y sus métodos de acción. DEBE extender PageHelpers para heredar las acciones con captura de evidencia (fillField, clickElement, asserts...).

El archivo completo

src/pages/LoginPage.ts

```
import { Locator, Page } from 'playwright';
import { PageHelpers } from '../PageHelpers';
import { IAttachFn, StepRecord } from '../../core/framework_actions/StepLogger';

const LOGIN_ROUTE = '/web/index.php/auth/login';

export class LoginPage extends PageHelpers {
  private readonly usernameInput: Locator;
  private readonly passwordInput: Locator;
  private readonly loginButton: Locator;
  private readonly sidebar: Locator;

  constructor(
    page: Page,
    attachFn?: IAttachFn,
    stepCounter?: { value: number },
    recordStep?: (record: StepRecord) => void,
  ) {
    super(page, attachFn, stepCounter, recordStep);
    this.usernameInput = page.getByPlaceholder('Username');
    this.passwordInput = page.getByPlaceholder('Password');
    this.loginButton = page.getByRole('button', { name: 'Login' });
    this.sidebar = page.locator('.oxd-main-menu');
  }

  async navigateTo(): Promise<void> {
    await this.navigateAndCapture(LOGIN_ROUTE, this.loginButton, 'Página de login cargada');
  }

  async fillUsername(value: string): Promise<void> {
    await this.fillField(this.usernameInput, value, 'campo Usuario');
  }

  async fillPassword(value: string): Promise<void> {
    await this.fillField(this.passwordInput, value, 'campo Contraseña', true);
  }

  async clickLogin(): Promise<void> {
    await this.clickElement(this.loginButton, 'botón Login');
  }

  async assertDashboard(): Promise<void> {
    await this.assertUrlContains('/dashboard/index', this.sidebar, 'Verifica dashboard');
  }
}
```

El constructor: por qué esos 4 parámetros

Ese constructor NO es opcional: es el "contrato" que el framework usa para crear tu página con captura de evidencia. El World lo llama así por ti (getPage).

Parámetro	Para qué sirve
page	La página de Playwright sobre la que actúas
attachFn	Adjunta las tarjetas HTML (screenshots) a la evidencia
stepCounter	Numera los pasos en la evidencia
recordStep	Guarda cada paso para el PDF de evidencia

- Copia el constructor TAL CUAL en cada Page Object y llama a super(page, attachFn, stepCounter, recordStep).
- Declara cada locator como `private readonly` y asígnalo en el constructor con `page.getBy...`
- Declara la ruta como una constante arriba del archivo (LOGIN_ROUTE).



IMPORTANTE — Debe extender PageHelpers

Si extiendes otra cosa (o nada), pierdes `fillField`, `clickElement`, los asserts y la captura de evidencia. Siempre: `class XPage extends PageHelpers`.

Cómo escribir tus métodos

Cada método usa un helper protegido de `BasePage/PageHelpers` (nunca llames `locator.fill` directo: usa `fillField`, así se captura la evidencia). Ejemplos:

métodos (patrón)

```
async fillUsername(value: string) { await this.fillField(this.usernameInput, value, 'Usuario'); }
async clickLogin()                  { await this.clickElement(this.loginButton, 'Login'); }
async assertDashboard()              { await this.assertUrlContains('/dashboard', this.sidebar, 'Dashboard'); }
```

3 Step Definitions

Conectan cada frase del feature con un método de tu página. Se obtiene la página con `this.getPage(LoginPage)` (el "this" es el World de Cucumber, que tiene la página lista).

El archivo completo

src/test/stepsDefinitions/login/LoginStepDefinitions.ts

```
import { Given, When, Then } from '@cucumber/cucumber';
import { CustomWorld } from '../../../support/world';
import { LoginPage } from '../../../pages/LoginPage';
import { JsonDataManagement } from '../../../core/data_management/JsonDataManagement';
import environments from '../../../core/settings/EnvironmentSettings';
import { LoginData } from '../../../core/interfaces/LoginData';

Given('el usuario está en la página de login', async function (this: CustomWorld) {
  await this.getPage(LoginPage).navigateTo();
});

When('ingresa el usuario del registro {string}', async function (this: CustomWorld, arg0: string) {
  const data = JsonDataManagement.getById<LoginData>(environments.env, 'login', arg0);
  await this.getPage(LoginPage).fillUsername(data.username);
});

When('ingresa la contraseña del registro {string}', async function (this: CustomWorld, arg0: string) {
  const data = JsonDataManagement.getById<LoginData>(environments.env, 'login', arg0);
  await this.getPage(LoginPage).fillPassword(data.password);
});

When('hace clic en iniciar sesión', async function (this: CustomWorld) {
  await this.getPage(LoginPage).clickLogin();
});

Then('el usuario es redirigido al dashboard', async function (this: CustomWorld) {
  await this.getPage(LoginPage).assertDashboard();
});
```

Reglas clave de los steps

- La frase entre comillas de Given/When/Then debe coincidir EXACTO con la del feature.
- Usa {string} en la frase para capturar un valor variable; llega como parámetro (arg0).
- this: CustomWorld te da getPage(). getPage cachea la página, así que puedes llamarlo varias veces.
- Para leer datos: JsonDataManagement.getByld<LoginData>(environments.env, "login", id).
- El segundo argumento ("login") es el nombre del archivo de datos: jsonData/<env>/login.json.



INFO — El valor entre comillas

En el feature escribes ...registro "valido". Ese "valido" llega como arg0. Con getByld lo usas como id para sacar el registro y pasar data.username / data.password al método.

4

Feature

El guion en Gherkin. Cada línea Given/When/Then debe tener su step definition (misma frase). Sin comillas raras: usa "comillas dobles" solo para los valores variables.

```
src/test/features/login/Login.feature
```

```
Feature: Login – Autenticación de usuarios
```

```
Scenario: Login exitoso con credenciales válidas
```

```
Given el usuario está en la página de login
```

```
When ingresa el usuario del registro "valido"
```

```
And ingresa la contraseña del registro "valido"
```

```
And hace clic en iniciar sesión
```

```
Then el usuario es redirigido al dashboard
```



IMPORTANTE — Los tags de Jira NO se ponen a mano

No escribas @jira:KEY en el feature. El sistema (QA Bridge) lo agrega automáticamente tras crear el caso en Jira. Ver docs/QA-BRIDGE-INTEGRATION-PROMPT.md.

Ejecutar

Correr la prueba

```
# No hay que registrar nada: Cucumber descubre los 4 archivos por convención
```

```
PS> cross-env HEADLESS=true npm run test:qa
```

```
# Solo tu feature:
```

```
PS> npm test -- src/test/features/login/Login.feature
```

Referencia: métodos que puedes llamar

Dentro de un Page Object (que extiende PageHelpers) tienes estos métodos protegidos. Úsalos en tus métodos; ellos capturan la evidencia automáticamente.

Acciones (BasePage)

Método	Qué hace
<code>fillField(loc, value, label?, masked?)</code>	Escribe en un campo (masked=true oculta el valor)
<code>clickElement(loc, label?)</code>	Hace clic
<code>selectOption(loc, value, label?)</code>	Elige opción de un <select>
<code>checkElement(loc, label?, checked?)</code>	Marca/desmarca checkbox o radio
<code>chooseRecord(loc, label?)</code>	Selecciona un registro/fila/autocomplete
<code>uploadFile(loc, paths, label?)</code>	Sube uno o varios archivos

Navegación y aserciones (PageHelpers)

Método	Qué hace
<code>navigateAndCapture(ruta, ancla, desc)</code>	Navega y espera a que el ancla sea visible
<code>navigateAndWaitForRedirect(ruta, patron, desc)</code>	Navega y espera una redirección
<code>assertVisible(loc, desc)</code>	Verifica que un elemento se ve
<code>assertLocatorText(loc, texto, desc)</code>	Verifica que un elemento contiene un texto
<code>assertAllTextsEqual(loc, texto, desc)</code>	Verifica que varios elementos tienen ese texto
<code>assertUrlContains(fragmento, ancla, desc)</code>	La URL contiene X y el ancla está visible
<code>assertUrlMatches(patron, desc)</code>	La URL coincide con un patrón/glob

Locators de Playwright (en el constructor)

Estrategia	Código
<code>getByRole('button', { name: 'X' })</code>	Por rol + nombre accesible (recomendado)
<code>getByPlaceholder('X')</code>	Campo por su placeholder
<code>getByLabel('X')</code>	Campo por su etiqueta
<code>getByText('X')</code>	Elemento por su texto
<code>getByTestId('X')</code>	Por data-testid
<code>locator('.mi-css')</code>	Selector CSS
<code>locator('xpath=//...')</code>	XPath



CONSEJO — Tip anti "strict mode"

Si un selector matchea varios elementos y solo quieres el primero, usa `.first()` en el locator. Los helpers de espera/ancla del framework ya lo hacen internamente.

Rutas de import (según dónde vive el archivo)

Desde	Import
Page (src/pages/)	'./PageHelpers' · '../core/framework_actions/StepLogger'
Page (src/pages/)	'../core/interfaces/XData' (si usas el tipo)
Steps (.../stepsDefinitions/mod/)	'../support/world' · '../pages/XPage'
Steps	'../core/data_management/JsonDataManagement'
Steps	'../core/settings/EnvironmentSettings' · '../core/interfaces/XData'

Los 3 tipos de escenario, a mano

Lo que arriba fue el tipo BÁSICO (un step por acción). Estos son los otros dos, escritos a mano.

Agrupar acciones en un step (a mano)

No hay "composite" en el código: es simplemente un método de tu página que llama a otros. Recibe el registro completo.

agrupar acciones

```
// En LoginPage.ts – un método que agrupa:
async iniciarSesion(data: LoginData): Promise<void> {
  await this.fillUsername(data.username);
  await this.fillPassword(data.password);
  await this.clickLogin();
}

// En los steps – un solo step para todo el formulario:
When('inicia sesión con el registro {string}', async function (this: CustomWorld, arg0: string) {
  const data = JsonDataManagement.getById<LoginData>(environments.env, 'login', arg0);
  await this.getPage(LoginPage).iniciarSesion(data);
});
```

Scenario Outline (a mano)

Corre el mismo escenario por cada fila de Examples. El step usa {string}; en el feature usas el placeholder <id>. Los valores del Examples los escribes tú (deben existir como id en tu login.json).

Login.feature (outline)

```
Scenario Outline: Login inválido por cada caso
  Given el usuario está en la página de login
  When inicia sesión con el registro "<id>"
  Then se muestra el error de credenciales inválidas
```

Examples:

id	
pass-malo	
usuario-malo	



INFO — A mano no hay filtro por status

El campo "status" que excluye filas es una función del generador. Escribiendo a mano, TÚ decides qué filas poner en la tabla Examples (agregas o quitas líneas).

Errores comunes al escribir a mano

Síntoma	Causa y solución
Undefined step / no ejecuta	La frase del feature no coincide EXACTO con la del step. Revisa texto, tildes y comillas.
Ambiguous / Multiple step definitions	Dos steps con la misma frase (en tu módulo u otro). Usa frases únicas por módulo.
No existe fillField / clickElement	La clase no extiende PageHelpers. Corrige: class XPage extends PageHelpers.
No captura evidencia	Llamaste locator.fill directo. Usa los helpers (fillField, clickElement...).
Cannot find module ...	Ruta de import mal. Revisa la tabla de rutas según la ubicación del archivo.
Record ... not found	El id del feature no existe en el json, o el nombre del archivo en getById no coincide.
Screenshot en blanco / timeout	Falta esperar la carga. Usa un ancla en navigateAndCapture y asserts con locator visible.



LISTO — Checklist final

1) datos + interface 2) Page Object extends PageHelpers con el constructor de 4 params 3) steps con las frases exactas 4) feature sin tags de Jira 5) cross-env HEADLESS=true npm run test:qa.